

Enhancing security and robustness of Cyphal on Controller Area Network in unmanned aerial vehicle environments

Kyusuk Han^{a,*}, Hyeran Mun^b, Malavika Balakrishnan^a, Chan Yeob Yeun^{b,*}

^a Secure Systems Research Center (SSRC), Technology Innovation Institute, United Arab Emirates

^b Center for Cyber-Physical Systems, Department of Electrical Engineering and Computer Science, Khalifa University, United Arab Emirates

ARTICLE INFO

Keywords:

Cyphal
UAVCAN
Security
Controller Area Network (CAN)
Authentication
Collision avoidance

ABSTRACT

A controller area network (CAN) has been widely deployed in automotive environments due to the decentralized environment which enables the easy control of many components in a vehicle. The CAN is now being adopted in other areas, including aviation and robots, especially unmanned aerial vehicles (UAV). Cyphal has been developed for years to provide communication over the CAN in UAV environments. For instance, the plug-and-play mode of Cyphal enables the automated configuration of the unique source IDs to nodes on the CAN bus. This reduces the overhead to assign static nodes to each node newly attached to the CAN bus. As such a configuration may occur the collision of the frames to the small size of the source ID, Cyphal currently recommends the use of such features only for non-critical applications. While attaching a new node to the CAN bus needs the authentication process to prevent the unauthorized devices attached and used by adversaries, Cyphal does not consider such security issues. To address these issues, we propose a security design to improve the robustness and security of the plug-and-play mode in Cyphal. Our proposal provides the collision avoidance and authentication of modules. To give efficiency in lightweight modules, our design uses the symmetric key-based method. We evaluate how the collision is avoided in our design compared to Cyphal, and also analyze the security of our proposal. Evaluation results demonstrate that our proposed scheme is effective in enabling the plug-and-play mode of Cyphal for the critical environment.

1. Introduction

A *Controller Area Network* (CAN) has been one of the most common networks for the automotive environment where multiple electronic control units (ECUs) are used. Since Rober Bosch GmbH proposed the specification of CAN in the early '80s (Robert Bosch GmbH, 1991), CAN became the *de facto* standard in the automotive industry and also has been standardized (ISO/TC 22/SC 31 Data communication, 2015). With the simplicity and the decentralized nature of CAN, connecting hundreds of ECUs over the CAN could be maintained without increasing the design complexity. Due to its robustness and convenience, CAN is being adopted in other cyber-physical system domains including aviation and robot. The unmanned aerial vehicles (UAVs) domain is not an exception to adopting CAN.

While the UAVs are emerging and integrating multiple components such as a flight controller, a mission computer, multiple sensors, and motors, the *Uncomplicated Application-level Vehicular Computing And Networking* (UAVCAN) has been designed to facilitate robust real-time intra-UAV communication between distributed components of UAVs. Implementation of UAVCAN has been done in several cases including *PX4 Autopilot*, which is an open-source autopilot project to provide a standard to deliver drone hardware support and software stack¹. UAVCAN has been rebranded as *Cyphal* (OpenCyphal Development Team, 2022) since 2022, and is being improved to provide the benefit that vendors could share the common communication protocols including certain data formats, while the older version (version 0) is also rebranded as *DroneCAN*² for legacy support.

Although it is obvious that adopting CAN into UAVs could give an advantage in designing various UAV architectures and managing com-

* Corresponding authors.

E-mail addresses: Kyusuk.Han@tii.ae (K. Han), hyeran.mun@ku.ac.ae (H. Mun), malavika.balakrishnan@tii.ae (M. Balakrishnan), chan.yeun@ku.ac.ae (C.Y. Yeun).

¹ <https://px4.io/>.

² <https://dronecan.github.io/>.

ponents in UAVs simpler, as it could do in automotive environments, there are several differences to be considered between domains. For example, unlike the automotive systems, in UAV scenarios, certain applications allow users to customize the UAV by replacing motor controllers or sensors. In this case, those components may not be from the original UAV manufacturers, and Cyphal specifies a *plug-in-play* mode that enables attaching new components and configures such custom components automatically. The communication of CAN is based on the ID of nodes, so all nodes on the CAN bus need their unique node IDs. A new custom node attached to the CAN bus may not have the node ID by default, so it needs to get a unique ID by requesting ID allocation to a component on the CAN bus, whose role is *ID allocator*. At this moment, the new node uses a *pseudo-random ID* to send the ID allocation request, as an anonymous *allocattee* since it doesn't have an allocated ID yet. This operation is called '*anonymous transfer*' in Cyphal.

However, the anonymous transfer could collide with high probability since the size of the pseudo-random ID is limited to 7 bits. Cyphal specification already addresses this issue and gives guidance on limiting the use of anonymous transfer such as using it only for a small number of components and only for non-critical applications.

In addition to the collision issue, security concerns rise by allowing the attachments of the custom components to the UAV. Since the components could be from multiple manufacturers/vendors, it could potentially happen that any unauthorized component is attached and used by the attacker. While Cyphal does not consider security in its scope, there have been various investigations of the cyber-physical threats on UAVs and much research work to provide security in CAN communication (Hartkopp et al., 2012; Bruni et al., 2014; Nürnberger and Rossow, 2016; Bella et al., 2019; Palaniswamy et al., 2020; Groza and Murvay, 2013; Wang et al., 2017; Jo et al., 2020; AUTOSAR, 2020; Wen et al., 2020a). Thus, as only authorized authentic devices should be attached, adopting ideas of security from previous work into Cyphal could be considered.

Most works considered the scenarios of CAN in automotive environments, where all ECUs in an automobile are manufactured by car manufacturers (i.e., OEMs) with their supply chains. Since OEMs could take all security-related controls such as component authentication and key management during the production stage, which is a huge difference from UAV scenarios. Also, by the nature of the car, the attachment of a new ECU only occurs when customizing certain components such as the audio video (AV) head unit, or replacing the new but the same component in controlled environments. Thus, various research works on authentication and key management on CAN in automotive considered the assumption that there are pre-defined unique IDs and associated pre-shared keys.

In contrast, it is not easy to expect either that node IDs are pre-defined or that keys are pre-shared using *plug-in-play* devices in customizable UAV environments, as having keys from all components without knowing which components could be installed in the UAV is unrealistic. Thus, adopting ideas of security in automotive into the scenario of Cyphal still has limitations in the key management process.

In this paper, we propose a design to enable a secure and robust plug-and-play mode for Cyphal/CAN. This paper makes the following major detailed contributions:

- Our proposed design increases the resiliency of collision avoidance in the configuration stage of Cyphal. By increasing the size of the pseudo-random ID, our design only invokes less than 0.8% when the ideal maximum number (i.e., 128) of *plug-and-play* nodes is configured at once.
- Our proposed design enables the authentication and key agreement between the main component in UAV and the newly attached plug-and-play component, which doesn't originally share any priorly shared information.
- Our proposed design provides forward secrecy to plug-and-play components in compromising a main component and prevents com-

munication failure by using old information from old configurations.

- We show the practicality and feasibility of our design by evaluating overhead in security operations and storage, and compatibility analysis with current Cyphal specification.

The remainder of the paper is organized as follows. Section 2 briefly overview CAN and Cyphal as preliminary information. Section 3 discusses the related works. We discuss security issues in Cyphal in Section 4. We propose secure protocols to enable the secure key agreement between plug-and-play devices and secure messaging in Section 5. We evaluate the proposed design in Section 6.2. Finally, we conclude this paper in Section 7.

2. Preliminaries

2.1. Controller Area Network (CAN)

In order to communicate over the CAN bus, an ECU essentially needs to consist of three parts: a *CAN transceiver*, a *CAN controller*, and a *microcontroller*. CAN transceiver is a dedicated device and is used to handle the physical layer of CAN communication. CAN controller is usually embedded in the microcontroller and handles the data link layer of CAN communication, microcontrollers may be different per application. Certain ECUs may use the high-performance application processor (AP) to handle various applications, while other ECUs may use the low-power or medium-power microcontroller unit (MCU) to perform a simple and dedicated operation. As depicted in Fig. 1, although ECUs (CAN Node A, B, and C) may consist of different capabilities of microcontrollers such as APs or low-power/medium-power MCUs, they are on top of the same architecture of the CAN controller and the CAN transceiver. Data communication over CAN is based on the *CAN frame*.

As shown in Fig. 2, CAN 2.0 A and CAN 2.0 B standard formats (Robert Bosch GmbH, 1991) utilize 11 bits and 29 bits for the source ID, respectively. A series of dominant (0) and recessive (1) bits consist of the CAN frame. The decision of the vehicle is controlled by the transmitted data, which is then shared between ECUs in order to ensure data consistency. The CAN frame as indicated in Fig. 2 is able to handle up to 8 bytes of data, and the data length code (DLC) describes the length of data. A unique ID in the CAN frame is the message identifier, which is not included in an address about the transmitter/receiver. The CAN frame also includes not only the priority of the data but also the meaning of the data. Inter-frame space (IFS) sized in 3 bits represents a separation of the CAN frame.

The CAN frame is able to detect errors and then remedy the conflicts through actions such as discarding a frame, re-transmitting a frame, or raising an error flag (Robert Bosch GmbH, 1991). If errors are detected on the CAN bus, an error flag is transmitted from receiving ECU to unable the erroneous messages. All ECUs connected with the CAN bus manage two types of error counters- transmit error counter (TEC) and receive error counter (REC). When the error occurs during the sender ECU, the TEC value of it increases by 8. When errors arise by receiver ECU, the value of REC for it increases by 1. All errorless transmission and/or reception decreases both values of TEC and REC by 1.

All ECUs first set up *error-active* mode with the value of TEC and REC maintained under 127. When errors occur during *error-active* mode, the recognized ECU reports other ECUs about errors by transmitting an active error flag, which composes six dominant (0) bits. The ECUs received the active error flag raise the *bit error* (i.e., a transmitted bit and an output bit on the CAN bus are not equal). Moreover, the ECU sent the active error flag experiences *sutff error* (i.e., sixth consecutive equal bit is detected). Any transmitted or received data are therefore terminated. As the values of TEC or REC will exceed 127, switching the mode from *error-passive* to error-active occurs. The receiver ECU transmits a passive error flag that contains six recessives (1) bits and 11 bits ITS (i.e., originally 3 bits). The ECU, which issued the passive error flag,

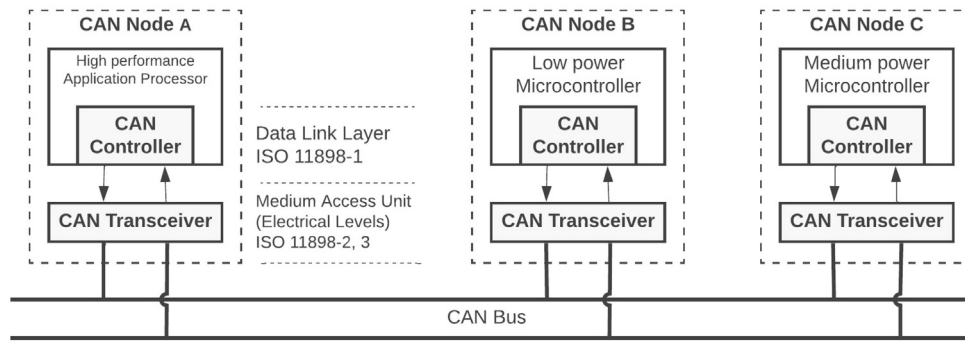


Fig. 1. An example CAN Node (ECU) connected to CAN Bus: Each CAN Node may have different capabilities, while they all commonly have CAN Controller and CAN Transceiver.

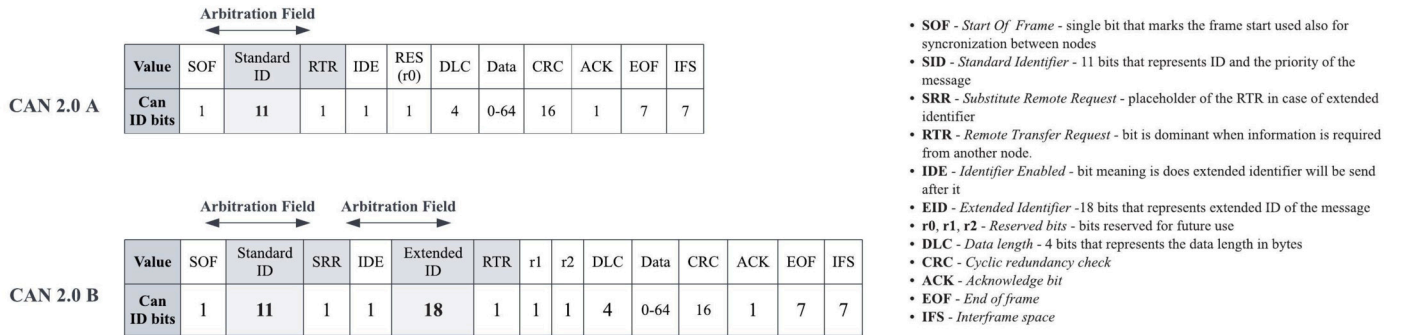


Fig. 2. CAN frame formats: CAN 2.0A standard format supports 11 bits for ID, and CAN 2.0B extended format supports 29 bits for ID field.

remains until the end of the frame since the consecutive six recessive bit is overwritten on the CAN frame. When the value of TEC is over 255, the status of the ECUs changes to *bus-off* mode. The ECUs under bus-off mode shut down and disconnect from the CAN bus because they ultimately serve as the cause of errors.

2.2. Cyphal

Cyphal is an open-source specification designed to provide intercommunication in the UAV. Cyphal is rebranded name from *Uncomplicated Application-level Vehicular Computing And Networking* (UAVCAN), which was originally designed to work on the CAN and is now being extended to support more types of networks including *user datagram protocol* (UDP) or Ethernet. As Cyphal is originally based on CAN, its design nature is also a decentralized peer network, where each peer communicates based on its identifier. Cyphal assumes that each device (i.e., CAN node/ECU) has a 128 bits unique identifier (UID), which is to be globally unique and never changed once assigned per hardware unit (OpenCyphal Development Team, 2022), running on top of the Microcontroller of the device.

2.2.1. Communication types in Cyphal

Cyphal employs broadcast and unicast transfers per purpose. A *broadcast transfer* does not define a specific destination, in contrast, a *unicast transfer* defines the specific destination to support the communication between the specific two nodes. While the CAN only has a broadcast nature, the unicast transfer of Cyphal is logically designed using the CAN message format.

Cyphal defines communications types as ‘*message*’ or ‘*service*’. For *message* transfer, the sender *publishes* the message, and one or multiple recipients subscribe to the message. The *message* transfer is only possible with a broadcast transfer. For *service* transfer, the sender requests the service and the recipient responds to it. Since the sender and recipient are explicitly assigned as depicted in Fig. 3 (b), the *service* transfer can be only a unicast transfer.

For each type of communication, a predefined set of data types is used, where each data type has a unique name. Message transfer uses *subject ID* to group the associated nodes with the same subject. Service transfer uses *service ID* to group the two nodes for the specific service. One notable type is *anonymous transfer*, which allows the broadcast transfer from the node without any assigned node ID, which is called the *anonymous node*.

2.2.2. Cyphal message format

Overall bit layouts of message formats of Cyphal are depicted in Fig. 3. Black-colored areas follow the generic CAN as discussed in Section 2.1.

Cyphal transport frames are defined based on CAN 2.0B, which uses a 29-bit extended CAN ID field, and defines two different bit layouts for message transfers as Fig. 3 (a) and service transfers as Fig. 3 (b). Moreover, the description of each common field is as below:

- **Priority** (3 bits): The priority is defined depending on the urgency.
- **Service, not Message** (1 bit): 0 and 1 indicate whether the frame is for the *message transfers* or *service transfers*, respectively.

Message transfer and service transfer formats are as below, respectively.

Message transfer format Message transfer format depicted in Fig. 3 (a) has the following fields.

- **Anonymous** (1 bit): 0 and 1 indicate regular message transfers and anonymous transfers, respectively. We discuss more detail in Section 2.2.3.
- **R0, R1, R1** (3 bits): Reserved bits. Set to 0, 1, 1, when transmitting.
- **Subject-ID** (13 bits): It indicates the ID of the current message transfer.
- **R0** (1 bit): Reserved bit. Set to 0.
- **Source node-ID** (7 bits): It is for the node-id of the message origin.

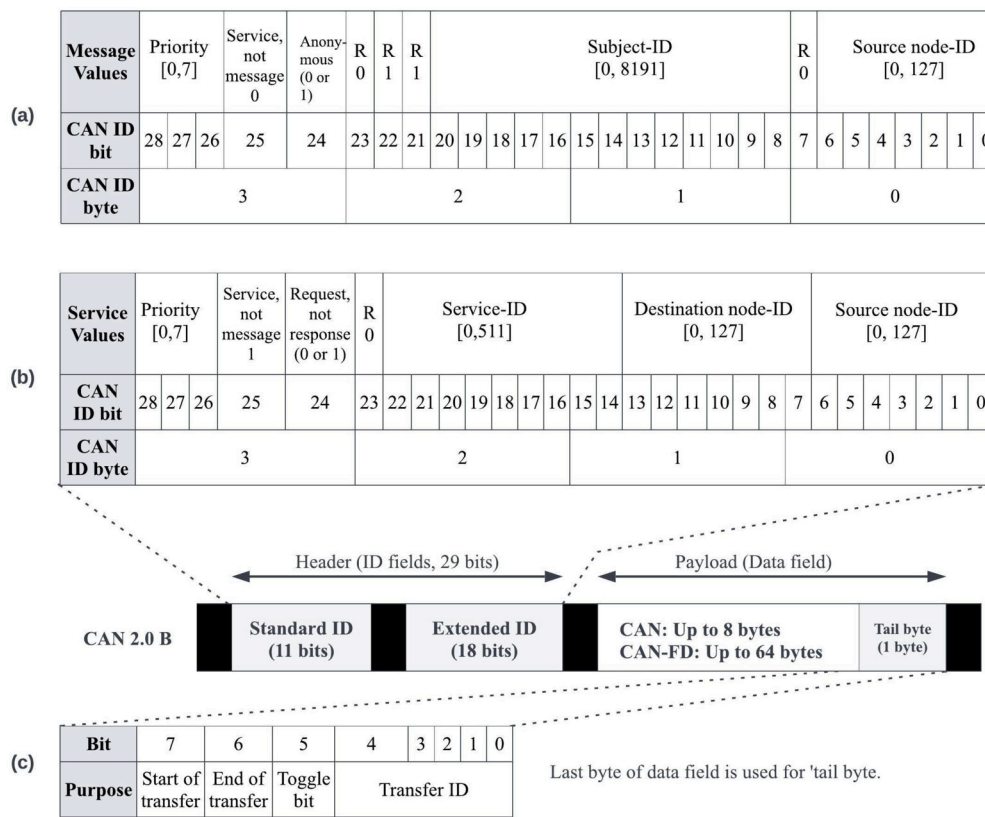


Fig. 3. Cyphal frame formats: (a) Message format for the data transfer, (b) Message format for the service transfer, and (c) Tail byte in payload.

Table 1

ID assignment in Subject-ID and Service-ID in Cyphal/CAN.

Subject-ID	Service-ID	Purpose
[0,6143]	[0,255]	Unregulated IDs.
[6144,7167]	[256,383]	Non-standard fixed regulated IDs.
[7168,8191]	[384,511]	Standard fixed regulated IDs.

When it is for anonymous transfer, the source ID field contains a pseudo-ID instead.

Service transfer format As shown in Fig. 3 (b), the service transfer format has the following fields.

- *Request, not response* (1 bit): 1 for a service request, 0 for a service response.
- *Service-ID* (9 bits): Service-ID of the encoded service object (request or response).
- *Destination node-ID* (7 bits): Node-ID of the destination - server if request, client in response.
- *Source node-ID* (7 bits): It is for the node-id of the origin.

Subject ID and service ID Each message subject is identified by a unique natural number – a *subject ID*; likewise, each *service* is identified by a unique *service ID*. The sets of subject-ID and service-ID are orthogonal. All identifiers are assigned to various functions, processes, or data streams within the network at the system definition time. Cyphal defines the ranges of Subject-ID and Service-ID as Table 1.

- *Unregulated identifiers*: Subject IDs between 0 and 6143, and Service IDs between 0 and 255, letting users assign the IDs.
- *Non-fixed identifiers*: Selected arbitrarily by a system integrator with changing relevant configuration parameters of connected nodes.

- *Fixed identifiers*: Governed by rules and regulations described in Cyphal specification.

Fixed identifiers can be used only with regulated data type definitions or with private definitions, and shall not be used with public unregulated data types, since that is likely to cause the identifier collision. All standard data type definitions in Cyphal specification are regulated. Therefore, Cyphal allows proprietary definitions with unregulated identifiers while also providing standard definitions commonly shareable across multiple stakeholders, which eases composing of multiple components by the user, from different vendors.

Data field Cyphal utilizes a fixed layout of the CAN data field that the last byte of the CAN data field contains for the metadata, referred to as the *tail byte* as shown in Fig. 3 (c). While the first three bits are used for multi-frame control, the last 5 bits of *transfer ID* are used for various purposes such as sequence monitoring, service response matching, transfer deduplication, or multi-frame transfer reassembly.

The preceding bytes of the data field contains the transfer payload. Since Cyphal does not use a remote frame defined in CAN, a CAN frame whose data field contains less than one byte is not a valid Cyphal/CAN frame.

2.2.3. Plug-and-play mode

Even if every Cyphal node shall have a node ID that is unique within the network, the newly attached node may not have the ID for the first time. Such IDs could be manually assigned by the UAV integrator or user. However, there exist circumstances where such manual assignment is either difficult or undesirable. For this case, Cyphal specifies the plug-and-play mode to allow the node attached to the CAN bus in UAV and join the Cyphal network and automatically obtain a node ID without any prior manual configuration. The plug-and-play mode requires the node to perform the anonymous transfer with Pseudo-random node ID.

Pseudo-random node ID Since the *plug-and-play* node does not have the node ID, it utilizes pseudo-random ID (pseudo-ID) for the 7-bit source node-ID field. As the CAN bus does not allow different nodes to transmit CAN frames with different data under the same CAN ID value, selecting the different random values between nodes is important.

Anonymous transfer Cyphal specifies the usage of the anonymous transfer for the *plug-and-play* mode. As depicted in Fig. 3 (a), CAN ID bit 25 is set to 1 for the anonymous transfer, and the source node-ID field of anonymous transfers shall be initialized with a pseudo-random pseudo-ID value.

Cyphal specification addresses the limitations for anonymous transfers only for the broadcast message as a single frame and also addresses that implementations shall account for possible errors on the CAN bus triggered by CAN ID collisions due to the small size of pseudo-ID values in anonymous transfer. Thus, Cyphal specification advises the use of anonymous transfers only for the *plug-and-play* feature for applications where high robustness is not critical.

3. Related work

In this section, we briefly overview the recent research works on securing CAN and then the security of UAV.

3.1. Security improvement of the CAN protocol

As vehicle technologies have been developing, they provide drivers with convenience and safety functions such as lane-keeping assistance and remotely turning on engines. For communication with several ECUs, the CAN - which is the *de facto* standard for in-vehicle networks - has been widely used.

However, the CAN bus not only increases interface channels with the external environment but also does not support security properties such as authentication and integrity of messages. This means that it has inadvertently encouraged cyberattacks and threats such as vehicle hacking (Koscher et al., 2010). Thus, automotive attacks have increased over the past few years by utilizing three types of attack surfaces such as physical access-based attacks (Miller and Valasek, 2013; Evenchick, 2015), wireless access-based attacks requiring initial physical access (Wen et al., 2020b,a), and wireless access-based attacks without physical access (Nie et al., 2017; Miller and Valasek, 2015). Especially, the authors described a bus-off attack that takes advantage of the CAN protocol's error-handling system. In a bus-off attack, an attacker periodically occurs CAN bus errors by injecting attack messages to force the specific ECU into bus-off mode. As a result, the attacker is left to monopolize the CAN bus and the legitimate ECUs are not able to take part in CAN communication (Cho and Shin, 2016).

To protect the CAN bus against attacks, several research groups have proposed schemes that generally use cryptographic primitives such as the message authentication code (MAC). The authors of (Hartkopp et al., 2012) suggested MaCAN to guarantee authentication and freshness of 4-byte messages by utilizing 4-byte MACs and timestamps. However, leaving one node unauthenticated during key establishment has been discovered (Bruni et al., 2014). Moreover, a variety of methods have indeed adopted group key-based authentication (Nürnberg and Rossow, 2016; Bella et al., 2019; Palaniswamy et al., 2020). In these approaches, there is no need to construct and then send several authentication tags that expand proportionately with the number of receivers because a single key is shared with all ECU (i.e., a group key). As a result, each CAN packet only generates one authentication tag. Furthermore, they are vulnerable to masquerade attacks that impersonate uncompromised ECUs. The various research works have applied centralized node-based MAC (Groza and Murvay, 2013; Wang et al., 2017; Jo et al., 2020). The advantages of these approaches are that they protect automotive CAN from masquerade attacks launched by infected ECUs since each ECU establishes its own authentication key with a centralized ECU. However,

these methods pose a potential risk regarding a single point of failure on the central node.

Security-enhanced CAN is also deployed at the industry level. Many automotive OEMs have already been proprietarily making their own authentication mechanism, as well as AUTOSAR (AUTomotive Open System ARchitecture) defines authentication mechanisms as the open and standardized software architecture (AUTOSAR, 2020). As depicted in Fig. 4, due to the constraints of CAN, vendors are utilizing truncated MAC such as 8, 16, or 32 bits, for up to 8 bytes of the payload of the CAN frame. The security strength depends on the vendor's requirements.

A variety of a number of wireless OBD-II dongles are developed, which are easy to plug into vehicles to enable remote functions such as vehicle control and status monitoring. However, OBD-II dongles emerge as a new attack surface for vehicles since these OBD-II are directly connected with in-vehicle networks and then transfer security parameters without any authentication. In (Wen et al., 2020a), the authors proposed an automated tool to dynamically test these dongles from all possible attack stages on a real automobile. As a result, 77 dongles expose at least two types of vulnerabilities.

Since most works addressed above focused on secure CAN in the automotive environment, applying them into the *plug-and-play* mode of Cyphal has limitations. In establishing the authenticated communication between nodes, there are situations that the modules may not share any prior secret. Therefore, the *plug-and-play* mode has to be considered to ensure secure communication in UAVs.

3.2. Research on UAV security

There have already been several communication methods for operating UAVs. In order to provide communication between the flight controller motors and sensors, several proprietary formats have been used over I2C, UART, and SPI, as well as communication over radio channels. Multiple communication protocols have been proposed to be used for UAV communication in (Khan et al., 2020a), micro air vehicle link (MAVLink) (Atoev et al., 2017), and Uranuslink (Kriz and Gabrlik, 2015).

MAVLink is one of the most commonly used protocols for bidirectional data communication between UAVs and ground control systems, and adopted widely into several open-source autopilot systems, including Ardupilot and PX4. While MAVLink did not have any security properties, which makes MAVLink vulnerable to several types of attacks, such as spoofing, message forgery, and denial of services (DoS). Although the improved MAVLink 2.0 includes signature support to prevent the forgery of messages, it is still vulnerable to eavesdroppers as data confidentiality is not provided. In (Allouch et al., 2019), the authors pointed out vulnerability in MAVLink and proposed a new protocol named MAVSEC by adding encryption to MAVLink to provide confidentiality.

Uranuslink (Kriz and Gabrlik, 2015) is a packet-oriented protocol that provides both unreliable and reliable services. It defines the packet structure as well as the data representation transmitted. The packet consists of the preamble, sequence number, message identification, data length, data field, and checksum. Uranuslink includes secured structures that provide both confidentiality and authenticity.

UAVCAN and a rebranded Cyphal are proposed to provide communication over the controller area network (CAN) for UAV communication. Cyphal³ is an open technology for real-time intra-vehicular distributed computing and communication based on modern networking standards for manned and unmanned aircraft, spacecraft, robots, and vehicles. Cyphal is being extended to run on CAN/CAN-FD, and multiple different transport-layer protocols (e.g. Ethernet, CAN, etc.). However, Cyphal does not consider security in its design scope.

³ <https://opencyphal.org/>.

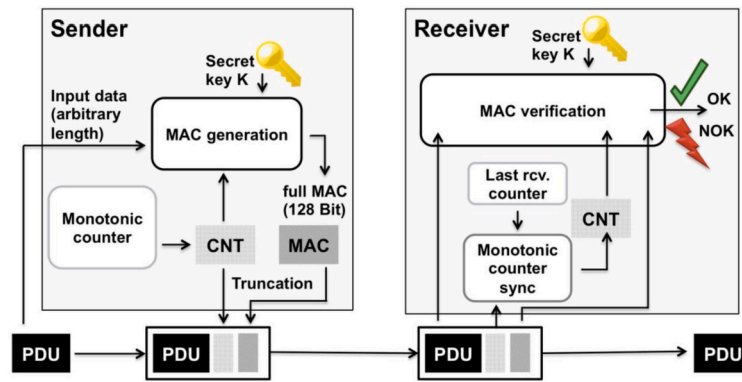


Fig. 4. AUTOSAR: Message Authentication and Freshness Verification for Secure On-board Communication (AUTOSAR, 2020).

Several efforts have been proposed to provide secure communications for UAVs by authentication and key management (Semal et al., 2018; Cho et al., 2020; Khan et al., 2020b; Lee et al., 2015/09; Abdel-Malek et al., 2021; Tanveer et al., 2020, 2021; Abdel-Malek et al., 2022; Jan and Khan, 2021; Khanh et al., 2020; Han et al., 2022). However, most works focused on communication over the radio channels, and none of them focus on the security related to UAVCAN or Cyphal.

Since studies discussed in Section 3.1 consider the automotive environment, they are not fit for UAV environments. Therefore, all the aforementioned schemes are not able to solve the security issues in Cyphal considering plug-and-play mode.

4. Security challenges in Cyphal

In this section, we discuss several challenges related to security and reliability in Cyphal. First, we discuss the potential collision issue *plug-and-play* mode addressed in the original specification in Section 4.1, then identify threats in considering the security of Cyphal.

4.1. Collision during anonymous transfer for ID allocation request

While Cyphal considers certain circumstances where all node IDs are manually managed as either difficult or undesirable, Cyphal advises using *anonymous transfer*, that temporarily uses the *pseudo-random ID* to send the static *ID allocation request*, thus each plug-and-play node can get a new static ID. However, due to the size of the source ID being only 7 bits, there is a high probability that attaching multiple components could invoke collision.

For example, only attaching 7 components could invoke above 10 percent of the probability of collisions as we analyze the cases in section 6.1. Thus, Cyphal advises using this *plug-and-play* mode only for the environment where the deterministic operations and robustness are not critical.

There are two ideas to avoid the collision in utilizing the anonymous transfer. The first way is to utilize vendor-specific *subject IDs* for the anonymous transfer instead of the fixed *regulated ID* for plug-and-play nodes (8155 and 8166). The second idea is limiting only one or two devices/modules that could use anonymous transfer (All other devices have static source ID). If the number of components is limited to only less than three, the collision could be less than 1 percent of the probability upon our analysis.

However, considering using Cyphal for more sophisticated applications, this limitation could be a critical drawback. Since sending a 7-bit random ID could already invoke the collision within the small number of modules, the increased complexity of the UAV using Cyphal will eventually make plug-and-play mode unusable.

Therefore, we need a fundamental method to avoid collision so as to preserve the reliability of the plug-and-play mode.

4.2. Security related threats in plug-and-play mode

It is obvious that allowing the connection of a new device without verifying its authenticity may cause potential security risks, as any adversary could use it as the attack path. Although Cyphal specification suggests the method to increase the integrity of the message in the *plug-and-play* mode by computing the arithmetic sum of all the transfer payload bytes and using the least significant bits of the result as the pseudo-ID (i.e., the use of stronger hashes is encouraged), still does not provide authenticity from a security perspective, as Cyphal addresses that security is not the scope of their specification.

Deploying the methods shown in section 3.1 would be considered to protect Cyphal by the authenticated communication over the CAN bus since most works are designed considering the automotive environments, they have a lack of consideration of establishing the authenticated communication for the *plug-and-play* device. Since it is not expected that a UAV contains keys of all devices predicting the future configuration, eventually we need to consider the method for verifying the authenticity of the source *ID allocation request* from the device that doesn't share any prior information.

In this condition, we identify the following three challenges to achieve the authenticity of plug-and-play mode.

4.2.1. Challenge 1. Joining unauthorized module in Cyphal network

Since the plug-and-play mode allows the situation that the devices are attached without prior configuration, any adversary module may attempt to be connected to the CAN bus and then get assigned a static source ID pretending a genuine module to exchange data. When the data frame is not verifiable, the adversary module may send malicious data to impact the operation of the UAVs.

To defend against it, the device authentication method as the plug-and-play mode should be allowed only for the authentic module without any prior shared secret. Once the modules are authenticated, any attempts of impersonation using the source ID by the adversary could be prevented by methods shown in Section 3.1.

4.2.2. Challenge 2. Disclosure of secret keys from compromised module

When the mutually authenticated modules are connected in the UAV, they could share the secret keys to enable authenticated communication. Since the modules are connected to a main module in the UAV, the adversary may compromise the main module to disclose the secret keys of all modules shared with the main module. Once the shared keys are disclosed to the adversary, the adversary may perform various attacks on the UAVs. For instance, the attacker may modify the source IDs of modules to invoke collisions which could impact the safety of the UAV.

Moreover, in addition to making the current system setup more vulnerable, the adversary may maliciously control the submodules no matter how the configurations are changed in new environments. Therefore, the system needs to have a method that prevents such attempts to

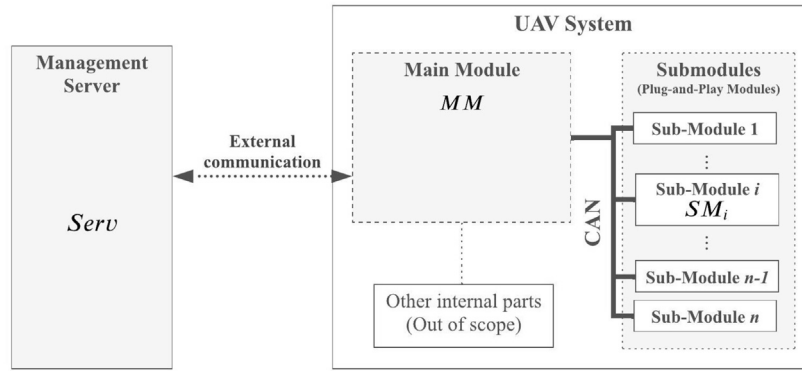


Fig. 5. System Model.

be continued once the compromised module is replaced or other genuine sub-modules have a new environment.

Thus, even if the adversary discloses the secret keys from the compromised main module, the impact shall be limited to the current composition of the UAV, the permanent secret in the other modules shall not be impacted and the forward secrecy shall be provided. The authentication mechanism requires forward secrecy that the secret keys shared between submodules shall not be usable anymore once the configuration is changed to the new environment, the adversary shall not have any advantage from the authentication process with the new environment.

4.2.3. Challenge 3. Connecting submodule to another UAV with colliding information

Once a submodule is authenticated, it may share the secret key establish cryptographically secure communication, and get assigned a static source ID. As the plug-and-play device, the submodules are possibly detached and reconnected to the other UAVs upon the UAV application scenarios.

In this case, if the submodule keeping old information, such as the assigned source ID and associated secret key, is connected to the new environment and that information is used, there could be a potential risk of bus error due to the colliding source IDs. An adversary may try to attempt to invoke such collision to lead the bus off attack (Cho and Shin, 2016).

Thus, there should be preventive ways to avoid such collisions, as well as to prevent the situation in that the adversary reuses other secret information to perform impersonation.

4.3. Design requirements

With the challenges and potential threats addressed above, we define the requirements as below.

- **Req1.Collision Avoidance:** The design shall provide proper ways to avoid collision in the ID allocation process for plug-and-play mode (Section 4.1) and also intentional behavior to lead collision (Section 4.2.3)
- **Req2.Authentication:** The design shall provide an entity and message authentication mechanism only allowing authorized devices can be used for plug-and-play mode (Section 4.2.1)
- **Req3.Forward Secrecy:** Disclosure of shared secret information from UAV's original component shall not affect the submodule used in new environments. (Section 4.2.2)
- **Req4.Compatibility:** The design should keep the compatibility to Cyphal.

5. Proposed design

In this section, we propose designs that enable collision avoidance in anonymous transfer and module authentication in *plug-and-play* mode, which improves the robustness and security of the Cyphal/CAN.

5.1. System environments

A UAV system consists of a main module and multiple sub-modules interconnected over the CAN bus. Note that the main module may also use different channels such as I^2C or UART, which is out of scope in this paper.

As shown in Fig. 5, each component is defined as follows:

- **Main Module MM :** A main component of UAVs and responsible for communication with other sub-modules over CAN. Furthermore, MM communicates with the server through the external channel. In *plug-and-play* mode, MM 's role is *allocator*, which allocates static source ID to *allocates*.
- **Sub Module SM_i :** A detachable sub-module used in UAV to perform only a specific operation such as temperature sensing or motor controlling. A UAV could consist of a n number of SM_i , where $1 \leq i \leq n$, connected to a CAN bus and performs only each specific operation. In *plug-and-play* mode, SM_i 's role is *allocatee*, which requests source ID from the *allocator*.
- **Management Server $Serv$:** Store shared keys with every sub-module SM_i . It communicates with the main module over the external communication channel, which is out of the scope of our paper.

As discussed in section 2.1, any component to be connected to the CAN bus requires CAN transceiver, CAN controller, and Microcontroller. Thus, as well as the main module MM , the submodule SM also needs to consist of a CAN controller and a CAN transceiver to be connected to the CAN bus. The microcontroller of SM is already capable of operations for CAN communication such as CRC32 computation. Moreover, as Cyphal/CAN specifies, SM needs to have the capability to store a unique device ID and to compute operations such as data truncation and random number generation.

However, although Cyphal specification prepares the certificate exchange to their specification, it may not expect to achieve every module in the UAV with the capabilities of PKI computation because many modules are lightweight to handle a dedicated operation. Thus, our design focuses on symmetric key-based authentication.

Therefore, we assume the following for our system model:

- A.1 All SM_i , where $1 \leq i \leq n$, are only connected to the CAN bus. Note that modules connected through other proprietary channels are out of the scope of our paper.

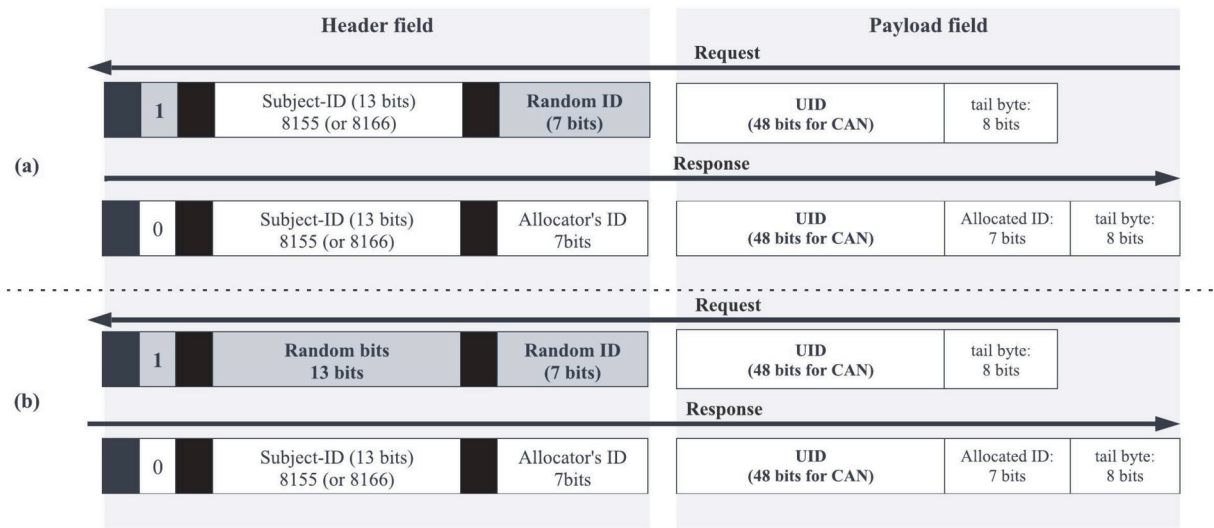


Fig. 6. (a) Allocating 7 bits in the source ID field by current Cyphal Specification (b) Allocating 20 bits as random bits using both subject ID field and source ID field. Note. Black-colored areas are unrelated fields in anonymous transfer.

- A.2 MM and SM_i has the microcontroller with the storage to keep the unique device ID and keys.
- A.3 MM and SM_i are capable of at least symmetric key-based crypto operations.
- A.4 Keys are not pre-shared among modules but are only shared with the management server $Serv$.

5.2. Avoiding the collision in ID allocation request

As discussed in Section 2.2.3, any module without a static ID has to perform the *anonymous transfer* to get the static ID. When the anonymous transfer is utilized, Cyphal specifies setting a *fifth* bit from the left in the header to '1', assigning *subject ID* 8165 and 8166 (for standard CAN), and using 7 bits of random ID as depicted in Fig. 6 (a).

Our finding is that the anonymous transfer is only used for the *ID allocation request* in *plug-and-play* mode. As a result, we identify that the use of a static subject ID is not necessary. Thus, we utilize the subject ID field for transmitting the randomized bits. Instead of using the static subject ID and 7 bits for the random ID in the current Cyphal specification, as depicted in Fig. 6 (a), we utilize a full 13 bits subject-ID field for the randomized bits along with the random ID. As shown in Fig. 6 (b), each frame for the anonymous transfer is able to use 20 bits of the random field. As a result, we could significantly minimize the possible collision compared to the original specification, and use this modification in secure plug-and-play mode.

5.3. Secure plug and play mode

Since we assume that the main module MM (i.e., *allocator*) does not share any key with n sub modules SM_i , $1 \leq i \leq n$ (i.e., *allocattee*) as discussed in Section 5.1, this process is to establish the security association between MM and SM_i .

Note that we omit the description of handling the CAN frame itself, as it follows the standard CAN message processing described in section 2.1.

5.3.1. Broadcasting random nonce

We design the random nonce distribution prior to ID allocation request. Here, we assume the specific subject ID is assigned to handle this random nonce distribution. As Cyphal doesn't specify this process yet, we can assign the ID between 0 and 6143 as unregulated ID, or between 6144 and 7167 as non-fixed ID as discussed in section 3.

- **Step 1.** The main module MM first broadcasts a random nonce *nonce*. For the first time, the counter in the tail byte is set to '0'.

The length of *nonce* could vary, but the maximum size available in a single frame on Cyphal/CAN is 56 bits. Based on the length of *nonce*, sending multiple frames could be considered, following the original rule in CAN and Cyphal. The counter in the tail byte increments whenever MM broadcasts the next random nonce. We define this random nonce distribution as a periodical procedure. For example, broadcasting random nonce every 1 second could be set. Note that setting the length of the interval is the implementer's choice. Recall that the random nonce distribution is not regulated in Cyphal specification yet, we consider using *unregulated ID* as shown in Table 1.

5.3.2. Initiation of ID allocation request

After receiving *nonce*, the sub-modules SM_i , where $1 \leq i \leq n$, initiate the node, called *allocattee*. Fig. 7 shows the overall process.

- **Step 2.** SM_i first retrieves UID_i and K_i which are a unique device ID and a shared key with $Serv$ respectively from the *storage*.

The size of UID_i could be 128 bits when the full bits are used, and it can also be used with 48 bits, which is a truncated size as Cyphal/CAN defines. 48 bits truncated ID is for the case of standard CAN, where the payload size is limited to only up to 8 bytes. Note that we omit the part of truncation of device ID in Fig. 7 to keep the description simple.

- **Step 3.** SM_i generates TB_i and RB_i ,

$$(TB_i, RB_i) = TR(HMAC(K_i, nonce || UID_i)),$$

where $HMAC(K, M)$ and $TR(M)$ denote the keyed hash of M with the key K and truncation of M , respectively. Truncation makes the truncated bits TB_i and remaining bits RB_i . For this case, we consider TB_i are 28 bits. RB_i is stored in the storage of SM_i .

- **Step 4.** SM_i splits 28 bits of TB_i into 13 bits TB_i^1 , 7 bits TB_i^2 , and 8 bits TB_i^3 .
- **Step 5.** SM_i performs ID allocation request, placing TB_i^1 and TB_i^2 to the subject ID field and source ID field in the CAN header respectively, and TB_i^3 to the payload.

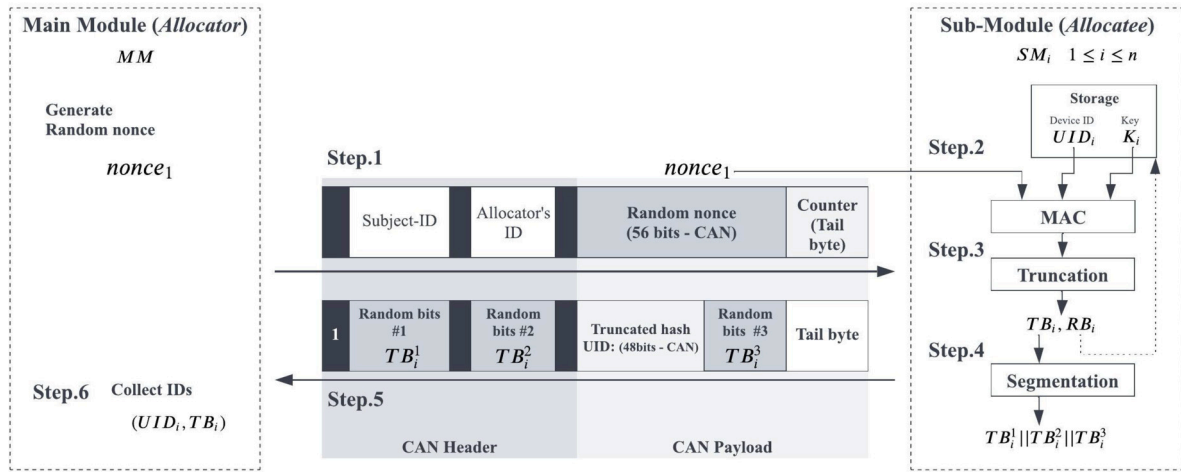


Fig. 7. The main module broadcasts random nonce and receives the unique IDs of sub-modules.

In receiving an anonymous CAN frame, which is the ID allocation request, by checking the anonymous field (i.e., CAN ID bit 24 is set to '1'), *MM* performs the following.

- **Step 6.** *MM* retrieves TB_i by concatenating TB_i^1 , TB_i^2 , and TB_i^3 , where $TB_i = TB_i^1 || TB_i^2 || TB_i^3$, and UID_i from the CAN frame.

5.3.3. Verification of requests from allocatees

Once the main module *MM* receives ID allocation requests from n number of sub-modules SM_i , $1 \leq i \leq n$, *MM* performs the following.

- **Step 7.** *MM* sends $nonce_1$, $\{(UID_i, TB_i) | 1 \leq i \leq n\}$ to the management server *Serv* through the external communication channel.

We assume that the external communication channel between the *allocator* and the management server is secure, and it is not our primary scope in this paper. Receiving data from *MM*, *Serv* checks each message from the *allocatee*.

- **Step 8.** *Serv* finds K_i of UID_i and generates TB_i^* and RB_i^* , where $(TB_i^*, RB_i^*) = TR(HMAC(K_i, nonce_1 || UID_i))$.
- **Step 9.** *Serv* checks if $TB_i^* \equiv TB_i$, to verify if the request from SM_i is valid.

If it is valid, *Serv* considers the requests from *allocatees* (i.e., n number of sub-modules SM_i , $1 \leq i \leq n$). *Serv* performs as follows:

- **Step 10.** *Serv* generates sek_i which is the session key to be shared between *MM* and SM_i , where $sek_i = KDF(K_i, RB_i^*)$. $KDF(M)$ denotes the key derivation function with the input M .
- **Step 11.** *Serv* sends $\{(UID_i, sek_i) | 1 \leq i \leq n\}$ to *MM* over the external channel. Note that we assume the external channel is secure and not our primary focus in this paper.

The overall processes between Steps 7 and 11 are depicted in Fig. 8.

5.3.4. Authenticated source ID allocation

As receiving $\{(UID_i, sek_i) | 1 \leq i \leq n\}$, *MM* performs as follows:

- **Step 12.** *MM* first selects the static source ID id_i to allocate to UID_i .

Note that the selection of the source ID depends on the implementer and follows the method from the original Cyphal specification.

- **Step 13.** *MM* generates a truncated message authentication code $auth_i$, where $auth_i = TR_1(HMAC(sek_i, UID_i || id_i))$. $TR_1(M)$ denotes the truncation of M .

- **Step 14.** *MM* generates truncated unique identifier $TUID_i$, where $TUID_i = TR_2(UID_i)$. $TR_2(M)$ denotes the truncation of M , which could use different method from TR_1 .

- **Step 15.** *MM* broadcasts $TUID_i$, id_i and $auth_i$ through the CAN bus. For this transmission, a subject ID is used.

SM_i checks the subject ID of the CAN frame and receives them to perform the following.

- **Step 16.** SM_i generates $TUID_i^*$ which is a truncated UID_i , $TUID_i^* = TR_2(UID_i)$, and checks if $TUID_i^* \equiv TUID_i$.

If the received truncated unique identifier $TUID_i$ is identical to $TUID_i^*$, SM_i performs the verification of the message as the following.

- **Step 17.** SM_i generates sek_i^* with K_i and RB_i which is stored after Step 3.

Note that it could be considered that generating sek_i^* before Step 16 without waiting for the response from *MM*.

- **Step 18.** SM_i generates $auth_i^*$, where $auth_i^* = TR_1(HMAC(sek_i^*, UID_i || id_i))$.

- **Step 19.** SM_i checks if $auth_i^* \equiv auth_i$.

If it is valid, SM_i considers $sek_i^* \equiv sek_i$ and uses sek_i as the shared key with *MM*. The overall processes are depicted in Fig. 9.

5.3.5. Secure communication between modules

When the key establishment process between *MM* and SM_i is completed at Step 19, both share sek_i . At this stage, the state becomes the same as the automotive environment. As applying for the existing work addressed in section 3.1 to enable authenticated communication is available, we omit the detailed scenario in this paper.

5.4. Device reconfiguration

In supporting the *plug-and-play* mode, it could also happen that the sub-module SM_i is detached after it is connected and assigned the static

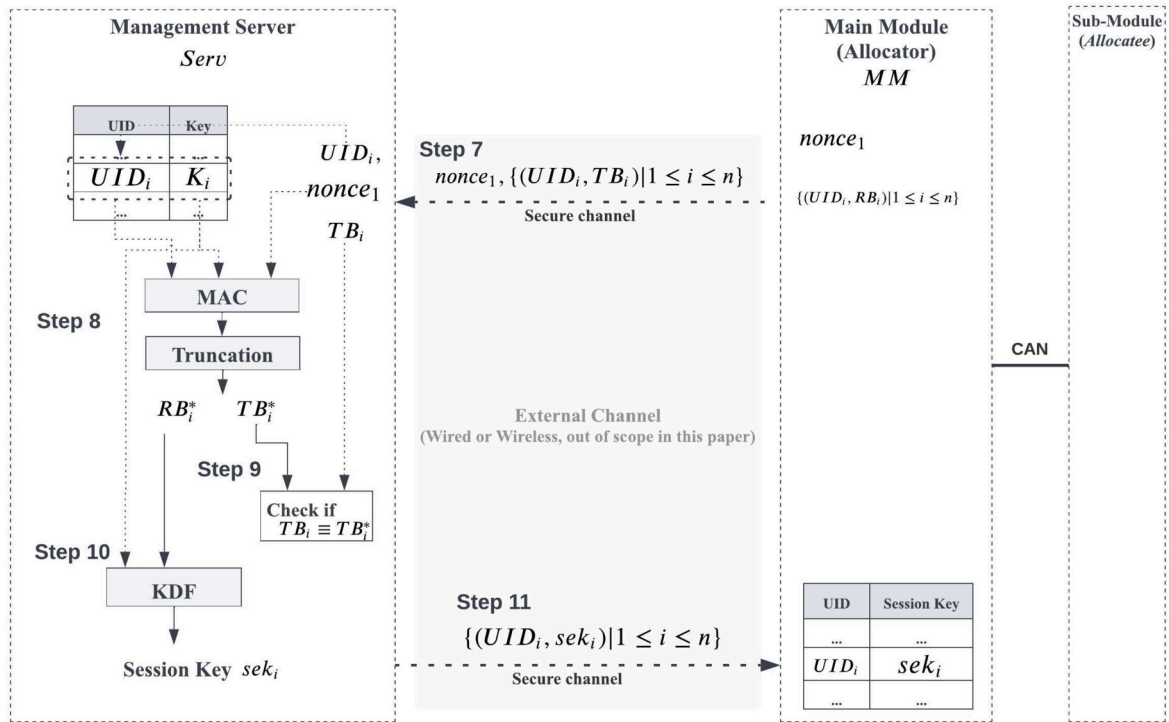


Fig. 8. Allocator and server exchange authentication information and receive the session keys to communicate with others.

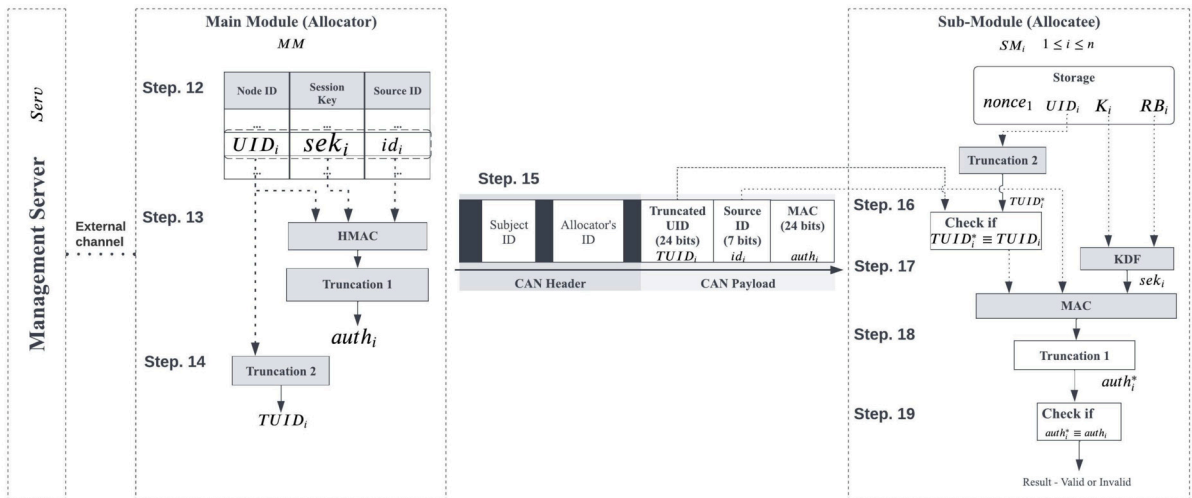


Fig. 9. Allocator and Allocatee exchange authentication information and receive the session keys to communicate with others.

ID. Here, we assume that the session key sek_i and the counter of the random nonce remain in SM_i and MM .

Let SM_i stores a counter ctr_j , where $ctr_j = ctr_{j-1} + 1$. The counter is incremented by 1. The counter uses 5 bits in the tail byte. When the device is turned on MM , it broadcasts the new random nonce $nonce_2$ with the continued counter ctr^* . Note that the random nonce is transmitted with a counter field using the payload 'tail' byte as depicted in Fig. 7.

As depicted in Fig. 10, SM_i performs the following during receiving the CAN frame.

- **Step 20.** SM_i checks if the counter ctr^* received from the CAN bus is identical to ctr_{j+1} which is incremented by 1 from the counter ctr_j that SM_i stores.

If ctr^* and ctr_{j+1} are not identical, SM_i removes the stored source ID and session key. Then SM_i performs the ID allocation request again.

When **Step 2** from **Step 19** are successfully completed, SM_i set counter ctr^* as the current counter ctr^j .

This reconfiguration mechanism is applied to the following scenarios.

- **Submodule without non-volatile memory:** Since the submodule SM_i does not store sek_i , ctr_j and id_i when the UAV is turned off, the reconfiguration step will bring the submodule always restart from **Step 2**. MM may always need to connect $Serv$ whenever the UAV is turned on.
- **Submodule accidentally disconnected while the UAV is turned on:** Although SM_i may store sek_i , ctr_j and id_i , disconnection may result in the mismatch of counters. In this case, retrying from initial authentication is required, and the reconfiguration step will also bring SM_i restart from **Step 2**.

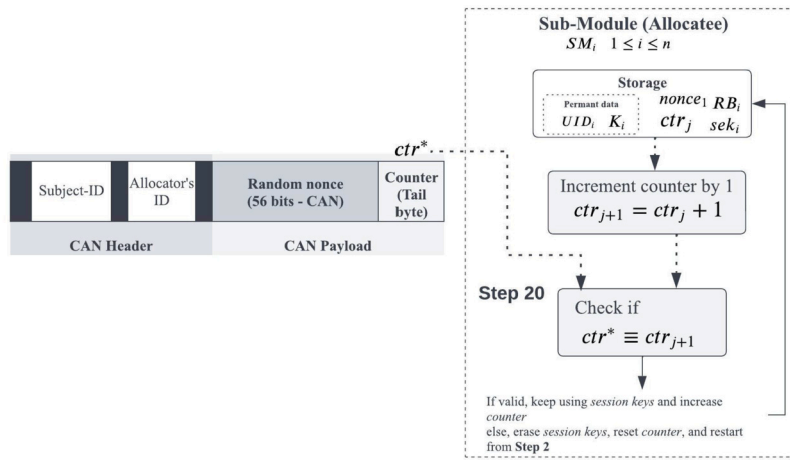


Fig. 10. Submodules use the counter to check the validity of the frame. If counters mismatch, reset the key and restart from Step 2.

- *Submodule disconnected while the UAV is turned off*: Attaching to the same UAV staying turned off may not affect states of sek_i , ctr_j and id_i stored in SM_i . However, attaching to the other UAV will result in the mismatch of counters, thus, the reconfiguration step will also bring SM_i restart from Step 2.

6. Evaluation

In this section, we evaluate the design. The collision avoidance in our proposed design is discussed in Section 6.1, and the security of the design is evaluated in Section 6.2. Finally, we demonstrate the potential overhead from adopting our design over the current Cyphal and discuss the practicality of the design on existing systems.

6.1. Improvement of collision-avoidance for anonymous transfer in ID allocation request

We show the design provides robustness to ID allocation requests with anonymous transfer in *plug-and-play* mode of Cyphal. Recall that Cyphal specifies a 7-bit source ID per device in the message header, and up to 128 different static source IDs could be assigned per module.

For the anonymous transfer, the current Cyphal only uses the 7-bit source ID field for the pseudo-random ID in the message header, thus up to 128 ($= 2^7$) different random IDs could be generated. In contrast, our proposed design utilizes the 13-bit subject field as the random bits, thus a total of 20 bits are used for the pseudo-random bits, and up to 1048576 ($= 2^{20}$) different unique IDs could be generated.

We could estimate the probabilities of collisions of random IDs in the current Cyphal and our proposal using the birthday problem as below equation (1).

$$\bar{P}(d, n) = 1 - \frac{(d)_n}{d^n}, \quad (1)$$

where $(d)_n = d \times (d-1) \times (d-2) \times \dots \times (d-n+1)$, and N samples can be chosen among d object. The probability of collision by the same source IDs from multiple devices is denoted as P . (i.e., $P = 1 - \bar{P}$). Assuming up to 128 different nodes could be attached, we measured the collision probabilities from attaching 1 sub-module to 127 sub-modules ideally.

As depicted in Fig. 11, Collision by using a 7-bit pseudo-random ID of Cyphal exceeds more than 50% ($P_{15} \geq 50\%$) when only 15 modules are attached in the *plug-and-play* mode. Collision exceeds 70% with 20 modules, and 90% with 26 modules. Attaching 36 modules already makes the probability of collision more than 99%. In contrast, our proposed design assigns 20-bit pseudo-random bits, and the probability of collision reaches approximately 0.005% for attaching 15 modules, 0.016% for 20 modules, 0.029% for 26 modules, and 0.057% for 36 modules. When the maximum possible number of modules (128

modules) is attached all together for the first time, the probability of the collision only becomes 8×10^{-3} , which is approximately 0.76% ($P_{128} \approx 0.76\%$), which is still lower than the probability of collision in attaching only 3 modules ($P_3 \geq 0.078\%$) in the current Cyphal specification.

Thus, our proposed design significantly improves the robustness of anonymous transfer in ID allocation requests and brings the opportunity that the *plug-and-play* mode can be used for critical scenarios, and we claim our design satisfies Req1 with the analysis in Section 6.2.3.

6.2. Security analysis in secured *plug-and-play* mode

We demonstrate that the proposed design mitigates the security challenges discussed in Section 4 and provides authentication for secure *plug-and-play* mode. As we already addressed security against physical attacks such as extracting the secret by disassembling the submodule is not our focus in this paper.

6.2.1. Security analysis against Challenge 1

As discussed in Section 4.2.1, an unauthorized sub-module may try to join the Cyphal network. Let a third-party attacker Adv attaches the malicious module SM^{Adv} to CAN bus to impersonate a genuine sub-module SM_i on the Cyphal network. Adv 's goal is to establish a secure communication channel to MM without valid authority. Adv may spoof the fraudulent ID allocation request or perform a replay attack by eavesdropping on the communication on the CAN bus.

Security against attacker's spoofing fake ID allocation request Pretending to be a genuine component, SM^{Adv} generates the malicious random bits without a valid key K_i and sends them to MM in Step 5. MM collects them and sends them to $Serv$ through Steps 6 and 7, and $Serv$ verifies the data generated by SM^{Adv} in Steps 8 and 9.

The probability of success that $Serv$ accepts the fraudulent bits TB^{Adv} from SM^{Adv} is not higher than the probability that Adv randomly chooses a valid string, where $TB^{Adv} \equiv TB_i$. In the case that the standard CAN is used, the probability of success could be $1/2^{28}$, as a total 28-bit size of random string could be used, with the constraint condition that the total payload of the CAN frame is only up to 8 bytes. In the case of using CAN-FD, more than 128-bit size of random string could be used, which gives the computationally infeasible security strength, as CAN-FD provides up to 64 bytes of payload.

If Adv succeeds in the attack at this stage, $Serv$ generates the session key sek^* at Step 10, using K_i with UID_i that Adv impersonates and the remaining bits RB^* , generated at Step 8. Once sek^* is given to MM , and MM will assign the source ID to the node with the message authentication code in Steps 13 to 15.

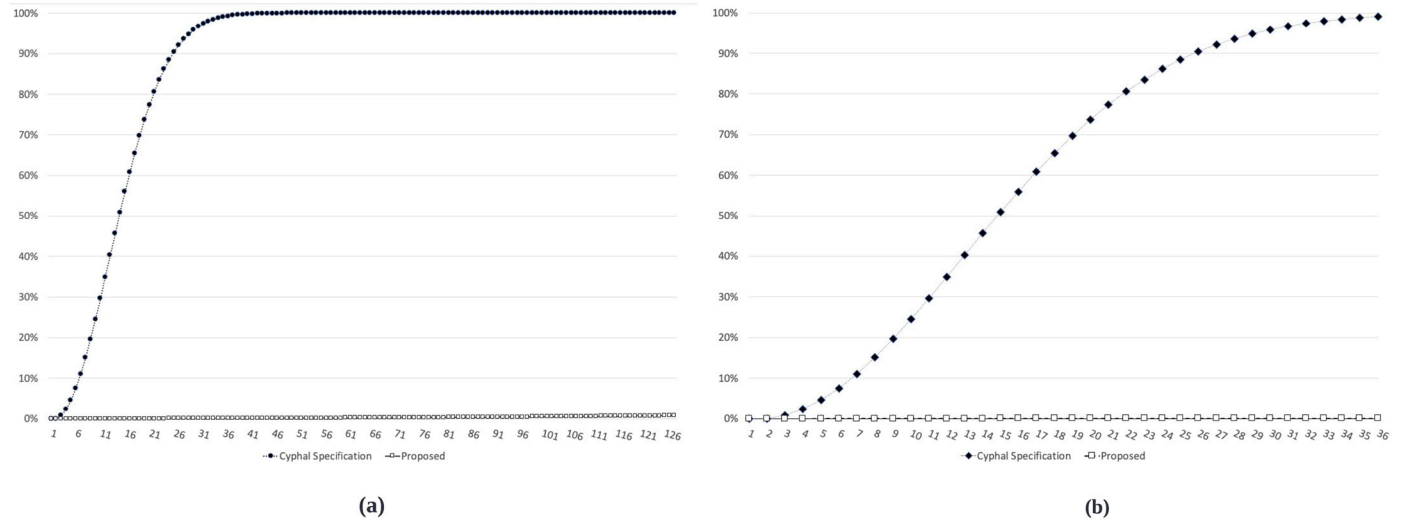


Fig. 11. Comparison of Probability of Collisions. The current Cyphal specification with 7 bits random ID, and the proposed design uses 20 bits random ID (a) Comparison up to 128 CAN nodes (b) Comparison up to 36 CAN nodes for highlight the difference.

Moreover, when the source ID is assigned, the scenario becomes the generic CAN security, and we can use the authentication methods of the CAN frame as discussed in Section 3.1. sm^{Adv} starts to send the fraudulent Cyphal frame to MM . Since the probability of success is not higher than the probability that Adv randomly chooses the key sek^{Adv} , where $sek^* \equiv sek^{Adv}$, which is negligible. Thus, sm^{Adv} needs to randomly generate the valid message authentication code without a valid key. The security of a single frame depends on the size of the message authentication code (MAC). Due to the constraint of the payload size of the CAN frame, assigning a smaller size (MAC) could be chosen as an implementation decision. For example, one byte could be assigned for MAC, whose probability of success is only $1/2^8$. However, even in this case, each attack is an independent event and the success of the attack doesn't affect to attack of the other frame, and continuing the attack success becomes more difficult since the attacker needs to generate the fake MAC with the same probability. Moreover, as we discussed in Section 3.1, the industry model (AUTOSAR 2020, Fig. 4) also adopts the truncated MAC, and our design doesn't decrease any security strength more than that. Using CAN-FD, which provides up to 64-byte payload, enables the full size of MAC which is computationally infeasible.

Security against replay attack Since CAN is a decentralized network, an attacker could physically attach his malicious module to the CAN bus and eavesdrop on the random bits TB_i and UID_i of sm^i . When Adv performs the replay attack, re-sending the same data only results in the repetition of Steps 6 to 15 at best. When the attack succeeds, Adv may only receives $TUID_i$, id_i , and $auth_i$, which are already known publicly. Thus, the success of the attack doesn't give any incentive to the attacker to generate the fake MAC that is considered to be valid to MM . Moreover, MM could simply prevent this attack such as checking the existence of the session key sek_i and UID_i of the SM_i .

When Adv performs the replay attack by modifying UID_i to the attackers ID UID^{Adv} , Adv still needs to generate a valid TB^{Adv} that is considered as a valid TB_i by MM , and also a valid message authentication code without knowledge of K_i . Thus, the probability of attack success becomes as same as the probability addressed in Section 6.2.1 and we claim our design satisfies Req2.

Security against joining unauthorized main module In contrast to the cases above, we can also consider that an unauthorized main module MM^{Adv} may try to join the Cyphal network, and attempt to establish a secure communication channel with other genuine sub-modules. However, MM^{Adv} needs to be authenticated by the management server $Serv$ through the external channel. We can utilize several existing methods

such as using TLS (Transport Layer Security), and its detail is not our focus in this paper.

6.2.2. Security analysis against Challenge 2

As discussed in Section 4.2.2, the attacker may try to know the permanent secrets K_i of SM_i from the disclosed secrets in the current setup to manipulate the SM_i freely, by compromising the main module as it allows communications through external wired or wireless channels. Note that submodules SM_i are only connected to the CAN bus as the assumption A.1.

A main module MM may have the session keys of all modules SM_i and also knows the module's unique ID UID_i , source ID id_i , and a random nonce used in the current integration $nonce_1$ in a UAV. Once Adv compromises MM , Adv may disclose entire secret inside of MM as the worst case. Moreover, Adv may also try to obtain the original key K_i . However, as K_i is never extracted from SM_i , Adv needs to perform the further attack only with the information MM knows.

Adv may try to know K_i from sek_i . However, the key derivation function KDF used to derive sek_i is a one-way function that has the property of *pre-image resistance*, which is defined as; Given a hash value h , it should be difficult to find any message m such that $h = \text{hash}(m)$. Therefore, finding K_i from sek_i is performing the pre-image attack, whose probability of success is as same as the probability of attacking a one-way function. The probability of success depends on the strength of KDF . As we assume KDF in use is secure, the probability of success in Challenge 2 is considered negligible.

There can be the case that Adv tries to regenerate new session keys with SM_i sek_i^{Adv} without communication with the management server $Serv$. In this case, Adv may also perform Step 8 to Step 10 without knowledge of the K_i . The probability of success is also the same as the probability discussed in Section 6.2.1. Thus, we claim our design satisfies Req3.

If the attacker attaches compromised sub-module SM^{Adv} to the CAN bus, SM^{Adv} may try to spoof the message, which is detected by checking sek_i . Note that changing source ID is not the Cyphal-specific attack, which is addressed in Section 3.1.

6.2.3. Security analysis against Challenge 3

As discussed in Section 4.2.3, there can be an attempt that interconnects the sub-modules that has the same id_i to invoke collision.

Let SM_A be a sub-module from the previous setup UAV_A , and SM_B be a submodule in UAV_B , and they have been assigned the same id_i from *plug-and-play mode*. ('A' and 'B' are arbitrarily used to distinguish

Table 2

Example of sizes of Parameters to be stored in the storage of a sub-module.

Parameter	Size	State	In Cyphal Specification	Crypto
UID	128 bits	Permanent	O	-
Unique key	128 bits	Permanent	-	ex) AES-128
Nonce	128 bits	Temporal	O	-
RB (Used for key seed)	228 bits	Temporal	-	ex) HMAC-SHA256
Counter	5 bits	Temporal	O	-
Session key	128 bits	Temporal	-	ex) AES-128

the environment.) Now UAV_A and UAV_B are turned off, SM_A is detached and reconnected to UAV_B .

At the initiation time, when UAV_B is turned on, MM_B will first broadcast the random nonce $nonce_j$ with the counter ctr_j^B . SM_A then checks whether $ctr_i^A \equiv ctr_j^B$. Note that i and j are also arbitrary sequence numbers. Only when $ctr_i^A \equiv ctr_j^B$, SM_i preserves id_i , and the probability is approximately 3.125%.

In such case, SM_A and SM_B now use the same id_i , thus sending two CAN frames with the same CAN ID to the CAN bus at the same time could result in the bus-off attack (Cho and Shin, 2016). However, this can be simply mitigated by letting the sub-module erase the stored data when the communication error occurs. Once the bus-off attack happens, both SM_A and SM_B restart from **Step 2** to request the static node IDs and establish the new session keys as addressed in section 5.4.

In contrast, if SM_A and SM_B have different *interval* to send the CAN frame, and both may be able to send the CAN frames with the same ID id_i avoiding the collision. However, since SM_A only knows the different session keys sek_A , the CAN frame SM_A sends is discarded by MM_B . Moreover, there is no shared secret between MM_B and SM_A , as a result, SM_A also knows that sek_A is not valid. If SM_A is owned by the adversary Adv , Adv may try to generate a fake CAN frame that is considered valid by MM_B without knowing sek_B . The probability of success depends on the length of the message authentication code (MAC) in the payload of the CAN frame. For example, if 2 bytes (16 bits) are assigned, the probability of success is approximately 0.00305%, while Adv needs to perform a brute-force attack. Therefore, our proposal mitigates the challenge in Section 4.2.3 and we claim our design satisfies **Req1** with the analysis in Section 6.1.

6.3. Compatibility analysis

We show that our protocol has compatibility with Cyphal without conflict. Our proposal requires one subject ID for random number distribution. This can be used from unregulated IDs between 0 and 6143, or non-standard fixed regulated IDs between 6144 and 7167. Of course, if random number distribution becomes the standard, it could be assigned as standard fixed IDs between 7168 and 8191, as in Table 1.

For anonymous transfer in ID allocation requests, when the fifth bit in the header field is set to 1 and subject-ID is a random number in our proposal, instead of a static 8155 (or 8166) for the ID allocation request. For now, we can set the rule excluding 8155 and 8166 to avoid collision and violation of the specification.

When the security is added, in Step 15, instead of adding 48 bits of Truncated UID into the payload, we add 24 bits of Truncated UID with 24 bits of MAC. Although we may not be able to use the same subject ID, which causes the mismatch in data handling, we can also assign another number between 0 and 7167 for 'secured ID allocation' instead of the current static ID 8155 or 8166. Thus, we show our protocol satisfies **Req.4**.

6.4. Performance evaluation

We show how much overhead is needed to apply our design to the perspective of the size of data and overhead from crypto operations. As discussed in section 2.1, any component to be connected to the CAN

Table 3

Crypto operations in the sub-module.

Crypto Function	Number of Operation
Message Authentication Code Generation	1
Message Authentication Code Verification	1
KDF	1

bus requires a CAN transceiver, a CAN controller, and a Microcontroller. Thus, it is reasonable to consider that sub-module SM also those parts as our assumption **A.1**. Moreover, Cyphal/CAN specification requires that the component has the capability to store a unique device ID as our assumption **A.2**. In fact, the microcontroller of SM is already capable of operations for CAN communication such as CRC32 computation and truncation of data, and may be capable of random number generation.

6.4.1. Overhead of storage requirements

To apply our design, the storage in SM should be capable of storing the following data. We show the example of the size of parameters in Table 2.

Storing all parameters requires 745 bits, while 261 bits are already required to be used in Cyphal. Lightweight microcontrollers usually have a size of storage (i.e. RAM, ROM, etc.) more than 32/64 KB, thus the storage size may not be a significant matter for this amount of overhead. Note that storing such confidential data in a protected environment inside of the module is not our focus in this paper.

6.4.2. Overheads of crypto operations

We show how much overhead is added from crypto operations. Although the main module may have more overhead by generating message authentication codes (MAC) for up to 128 sub-modules connected on the CAN bus and also the communication to the management server, we assume that the main module has more computing capabilities, as handle those operations may not be an issue. Thus, we focus on the computation overhead in the sub-module in Table 3. Two operations of MAC generation and verification and key derivation are added. (Note that overheads from simple operations such as truncation or segmentation are negligible.)

However, adding such operations does not invoke huge overhead to the sub-module, as several experiments (Grisafi et al., 2022; Silva Junior et al., 2018) show the comparison of cases using CRC32 and SHA256, both are done in far less than 0.5 ms for handling 64 bytes data.

Several existing commercially available low-power entry-level microcontrollers have the capabilities to handle such crypto operations. For example, cryptographic performance on STM32L0 series from *STMicroelectronics* shows 308 *microseconds* for SHA256 computation. Moreover, such operations are happening during the initiation process for ID allocation in *plug-and-play mode*, those overheads do not give any impact on the drone in flight.

Although the maximumly possible overhead for the main module is 128 * (overhead of SM) on the CAN bus with the overhead from the communication to the management server, the main module is generally a high-power microcontroller, which does not become any meaningful burden in addition to the process is not a time-critical case including ID allocation request. Another overhead comes with the communication to the server, which depends on the characteristics of the

channel (wired or wireless) which is beyond of our scope of securing Cyphal/CAN.

7. Conclusion

Cyphal is designed to support communications over the Controller Area Network (CAN) for UAV environments, as CAN is being adopted in various areas beyond automotive systems due to its simplicity and robustness. However, although certain features such as the anonymous transfer and the *plug-and-play* mode are defined to enable the auto-configuration of modules without static node IDs in Cyphal, using those for critical applications is not recommended due to the potential collision issue using CAN. Moreover, the lack of consideration for security could obstruct the wide usage of Cyphal. In this paper, we propose a novel design to enhance the robustness and security of Cyphal, especially in anonymous transfer and *plug-and-play* mode. Our design enhancement provides collision avoidance when the maximum possible modules are connected simultaneously, and also provides device authentication for the new device in anonymous transfer for the *plug-and-play* mode. Our evaluation results show that our novel enhancement enables the *plug-and-play* mode of Cyphal even for the critical environment. In our future work, we suggest the proposed design to Cryphal specification and integrate it into Cyphal standard data type and implementation.

CRedit authorship contribution statement

Kyusuk Han: Conceptualization, Investigation, Methodology, Writing – original draft. **Hyeran Mun:** Investigation, Writing – review & editing. **Malavika Balakrishnan:** Resources, Software. **Chan Yeob Yeun:** Funding acquisition, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- OpenCyphal Development Team, 2022. Cyphal Specification 1.0 (beta). Tech. Rep., OpenCyphal.org.
- Abdel-Malek, M.A., Akkaya, K., Bhuyan, A., Ibrahim, A.S., 2021. A proxy signature-based drone authentication in 5G D2D networks. In: 2021 IEEE 93rd Vehicular Technology Conference (VTC2021-Spring), ieeexplore.ieee.org, pp. 1–7.
- Abdel-Malek, M.A., Akkaya, K., Bhuyan, A., Ibrahim, A.S., 2022. A proxy signature-based swarm drone authentication with leader selection in 5G networks. *IEEE Access* 10, 57485–57498.
- Allouch, A., Cheikhrouhou, O., Koubâa, A., Khalgui, M., Abbes, T., 2019. MAVSec: securing the MAVLink protocol for Ardupilot/PX4 unmanned aerial systems. In: 2019 15th International Wireless Communications & Mobile Computing Conference (IWCMC). IEEE, pp. 621–628.
- Atoev, S., Kwon, K.-R., Lee, S.-H., Moon, K.-S., 2017. Data Analysis of the MAVLink Communication Protocol, in: 2017 International Conference on Information Science and Communications Technologies (ICISCT). IEEE, pp. 1–3.
- AUTOSAR, 2020. Specification of Secure Onboard Communication Protocol. Tech. Rep. 969, R20/11, AUTOSAR.
- Bella, G., Biondi, P., Costantino, G., Matteucci, I., 2019. TOUCAN: a proTocol tO seCure Controller Area Network. In: Proceedings of the ACM Workshop on Automotive Cybersecurity. AutoSec '19. Association for Computing Machinery, New York, NY, USA, pp. 3–8.
- Bruni, A., Sojka, M., Nielson, F., Nielson, H. Riis, 2014. Formal Security Analysis of the MaCAN Protocol. *Lect. Notes Comput. Sci.*, vol. 8739, pp. 241–255.
- Cho, G., Cho, J., Hyun, S., Kim, H., 2020. SENTINEL: a secure and efficient authentication framework for Unmanned Aerial Vehicles. *NATO Adv. Sci. Inst. Ser. E Appl. Sci.* 10 (9), 3149.
- Cho, K.-T., Shin, K.G., 2016. Error handling of in-vehicle networks makes them vulnerable. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 1044–1055.
- Evenchick, E., 2015. An introduction to the CANard toolkit. In: Blackhat Conference, pp. 1–8.
- Grisafi, M., Ammar, M., Roveri, M., Crispo, B., 2022. PISTIS: trusted computing architecture for low-end embedded systems. In: 31st USENIX Security Symposium. USENIX Security '22, pp. 3843–3860.
- Groza, B., Murvay, S., 2013. Efficient protocols for secure broadcast in controller area networks. *IEEE Trans. Ind. Inform.* 9 (4), 2034–2042.
- Han, K., Nuaimi, E.A., Blooshi, S.A., Psiakis, R., Yeun, C.Y., 2022. A new scalable mutual authentication in fog-edge drone swarm environment. In: Information Security Practice and Experience. Springer International Publishing, pp. 179–196.
- Hartkopp, O., Reuber, C., Schilling, R., 2012. MaCAN-message authenticated CAN. In: 10th Int. Conf. on Embedded Security in Cars (ESCAR 2012).
- ISO/TC 22/SC 31 Data communication, 2015. Controller Area Network (CAN) Specification Version 2.0ISO 11898-1:2015 Road vehicles — Controller Area Network (CAN) — Part 1: Data link layer and physical signalling. Tech. Rep., International Organization for Standardization (ISO).
- Jan, S.U., Khan, H.U., 2021. Identity and aggregate signature-based authentication protocol for IoD deployment military drone. *IEEE Access* 9, 130247–130263.
- Jo, H.J., Kim, J.H., Choi, H.-Y., Choi, W., Lee, D.H., Lee, I., 2020. MAAuth-CAN: masquerade-attack-proof authentication for in-vehicle networks. *IEEE Trans. Veh. Technol.* 69 (2), 2204–2218.
- Khan, M.A., Ullah, I., Nisar, S., Noor, F., Qureshi, I.M., Khanzada, F.U., Amin, N.U., 2020b. An efficient and provably secure certificateless Key-Encapsulated Signcryption scheme for Flying Ad-hoc Network. *IEEE Access* 8, 36807–36828.
- Khan, N.A., Jhanjhi, N.Z., Brohi, S.N., Nayyar, A., 2020a. Chapter three - emerging use of UAV's: secure communication protocol issues and challenges. In: Al-Turjman, F. (Ed.), *Drones in Smart-Cities*. Elsevier, pp. 37–55. <https://www.sciencedirect.com/science/article/pii/B9780128199725000033>.
- Khanh, T.D., Komarov, I., Don, L.D., Iureva, R., Chuprov, S., 2020. TRA: effective authentication mechanism for swarms of unmanned aerial vehicles. In: 2020 IEEE Symposium Series on Computational Intelligence (SSCI), pp. 1852–1858.
- Koscher, K., Czeskis, A., Roesner, F., Patel, S., Kohn, T., Checkoway, S., McCoy, D., Kantor, B., Anderson, D., Snacham, H., Savage, S., 2010. Experimental security analysis of a modern automobile. In: Proceedings - IEEE Symposium on Security and Privacy, pp. 447–462.
- Kriz, V., Gabrlik, P., 2015. UranusLink - communication protocol for UAV with small overhead and encryption ability. *IFAC-PapersOnLine* 48 (4), 474–479.
- Lee, Y., Seok, Kim, E., Kim, Y. sub, Seol, D. chul, 2015/09. Effective message authentication method for performing a swarm flight of drones. In: Proceedings of the 2015 International Conference on Environmental Engineering and Remote Sensing. Atlantis Press, pp. 95–97.
- Miller, C., Valasek, C., 2013. Adventures in automotive networks and control units. In: Def Con 21 (260–264), pp. 15–31.
- Miller, C., Valasek, C., 2015. Remote exploitation of an unaltered passenger vehicle. In: Black Hat USA 2015 (S 91), pp. 1–91.
- Nie, S., Liu, L., Du, Y., 2017. Free-fall: hacking tesla from wireless to CAN bus. Briefing, In: Black Hat USA 25, pp. 1–16.
- Nürnberg, S., Rossow, C., 2016. vatiCAN – Vetted, Authenticated CAN Bus. Lecture Notes in Computer Science. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 106–124.
- Palaniswamy, B., Camtepe, S., Foo, E., Pieprzyk, J., 2020. An efficient authentication scheme for intra-vehicular controller area network. *IEEE Trans. Inf. Forensics Secur.* 15, 3107–3122.
- Robert Bosch GmbH, 1991. Controller Area Network (CAN) Specification Version 2.0. Tech. Rep., Robert Bosch GmbH, Postfach 300240, D-7000 Stuttgart 30.
- Semal, B., Markantonakis, K., Akram, R.N., 2018. A certificateless group authenticated key agreement protocol for secure communication in untrusted UAV networks. In: 2018 IEEE/AIAA 37th Digital Avionics Systems Conference (DASC), pp. 1–8.
- Silva Junior, E.A., Araujo-Filho, P.F.d., Campelo, D.R., 2018. Experimental evaluation of cryptography overhead in automotive safety-critical communication. In: 2018 IEEE 87th Vehicular Technology Conference. VTC Spring, pp. 1–5.
- Tanveer, M., Zahid, A.H., Ahmad, M., Baz, A., Alhakami, H., 2020. LAKE-IoD: lightweight authenticated key exchange protocol for the Internet of drone environment. *IEEE Access* 8, 155645–155659.
- Tanveer, M., Khan, A.U., Kumar, N., Hassan, M.M., 2021. RAMP-IoD: a robust authenticated key management protocol for the Internet of drones. *IEEE Int. Things J.*, 1339–1353.
- Wang, E., Xu, W., Sastry, S., Liu, S., Zeng, K., 2017. Hardware module-based message authentication in intra-vehicle networks. In: Proceedings of the 8th International Conference on Cyber-Physical Systems. ICCPS '17. Association for Computing Machinery, New York, NY, USA, pp. 207–216.
- Wen, H., Chen, Q.A., Lin, Z., 2020a. Plug-N-Pwned: comprehensive vulnerability analysis of OBD-II dongles as a new over-the-air attack surface in automotive IoT. In: 29th USENIX Security Symposium (USENIX Security '20). USENIX Association, pp. 949–965. <https://www.usenix.org/conference/usenixsecurity20/presentation/wen>.

Wen, H., Zhao, Q., Chen, Q.A., Lin, Z., 2020b. Automated cross-platform reverse engineering of CAN bus commands from mobile apps. In: *Proceedings 2020 Network and Distributed System Security Symposium. NDSS'20*, pp. 1–17.

Dr. Kyusuk Han is currently the principal researcher at Technology Innovation Institute (TII) in UAE. He studied cryptography and information technology and holds an M.S. (2004) and Ph.D. degree (2010) in information and communication engineering from the Korea Advanced Institute of Science and Technology (KAIST). After his Ph.D., he studied automotive cybersecurity, working as a visiting scholar and Research Fellow at the University of Michigan, Ann Arbor. Since late 2015, he was the chief research engineer in the CTO division of LG Electronics, working on security technology for the vehicle component, including software update security. During his work at LG Electronics, he focused on adopting the cybersecurity engineering process into the research and development operations and lectured to spread the process across the organizations. He is actively participating in standardization activities with various international standardization bodies including the Society of Automotive Engineers International (SAE) and RISC-V. Since his joining TII in 2021, his current research interest is achieving the security of Cyber-Physical systems based on a hardware-protected security environment.

Dr. Hyeran Mun received a B.S. degree in information security and computer engineering from Seoul Women's University, Seoul, Korea in 2007 and an M.S. in information and communication engineering from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, Korea in 2009. She was a researcher for the Electronics and Telecommunications Research Institution (ETRI), Daejeon, Korea. She received a Ph.D. degree from the Graduate School of Information Security, Korea University, Seoul, Korea in 2022. She is currently a visiting researcher at Khalifa University in UAE. Her research interests include vehicular-IT security and AI security.

Malavika Balakrishnan is currently working in Technology Innovation Institute in UAE, as a Senior Associate – Software Engineer. She studied Computer Science and Engineering at the College of Engineering, Guindy, Anna University, Chennai, in 2017. Malavika worked in various areas including the Kernel side of the operating systems, software development, and machine learning in Juniper Networks, KLA Tencor, and Friendly. Since her joining TII in 2023, Malavika has been actively involved in system configuration, analysis, and testing for various platforms.

Dr. Chan Yeob Yeun received the M.Sc. and Ph.D. degrees in information security from the Royal Holloway, University of London, in 1996 and 2000, respectively. After his Ph.D., he joined Toshiba TRL in Bristol, U.K., and later became the Vice President at the LG Electronics, Mobile Handset R&D Center, Seoul, South Korea, in 2005. He was responsible for developing mobile TV technologies and related security. He left LG Electronics in 2007, and joined ICU (merged with KAIST), South Korea, until August 2008, and then the Khalifa University Science and Technology, in September 2008. He is currently a Researcher in cybersecurity, including IoT/USN security, cyber-physical system security, cloud/fog security, and cryptographic techniques as an Associate Professor with the Department of Electrical Engineering and Computer Science, and an Active Member of Electrical Engineering and Computer Science, and active Member of the Center for Cyber-Physical System (C2PS). He also enjoys lecturing for M.Sc. Information Security and Ph.D. Engineering courses at Khalifa University. He has published more than 140 journal articles and conference papers, nine book chapters, and ten international patent applications. He also serves on the editorial board of multiple international journals and on the steering committee of international conferences.